

# **Taktill POS: A Cloud-Native Point-of-Sale System for Retail Businesses**

## **A PROJECT REPORT**

**Submitted by**

**[STUDENT NAME(S) / REGISTRATION NUMBER]**

*in partial fulfillment for the award of degree of*

**BACHELOR OF SCIENCE IN**

**[DEGREE PROGRAMME]**



**FACULTY OF INFORMATION AND  
COMMUNICATION TECHNOLOGY**

**ISBAT UNIVERSITY**

**A CHARTERED UNIVERSITY**

**JUNE 2026**

**INTERNATIONAL BUSINESS SCIENCE AND TECHNOLOGY  
UNIVERSITY  
JUNE 2026**



**BONAFIDE CERTIFICATE**

Certified that this project report titled **Taktill POS: A Cloud-Native Point-of-Sale System for Retail Businesses** is the bonafide work of [NAME OF STUDENT] who carried out this project under my supervision.

**HEAD OF DEPARTMENT**

.....

**FACULTY IN CHARGE**

.....



## DECLARATION

I **[NAME OF STUDENT]** do hereby declare that to the best of my knowledge, the matter embodied in this project report is original, and has not been submitted for any academic award by any student to any other University or Institution for the award of Degree.

**SIGNATURE**

.....

**DATE**

.....

## **ACKNOWLEDGMENT**

First and foremost, I would like to acknowledge my deepest gratitude to the Almighty God for the blessings, guidance, protection, and grace throughout this academic endeavour.

I would like to thank my supervisor, [Supervisor Name], for the valuable guidance, support, and constructive feedback that made the completion of this project possible. My sincere appreciation also goes to the faculty and staff of the Faculty of Information and Communication Technology at ISBAT University.

I equally express my special thanks to my parents and guardians for their moral and financial support throughout my academic journey.

Finally, I am grateful to my friends and course mates whose efforts and encouragement towards completing this academic project cannot be underestimated.

## **ABSTRACT**

This report presents the design, implementation, and evaluation of Taktill POS, a cloud-native, web-based Point-of-Sale (POS) system tailored for small and medium-sized retail businesses in East Africa. The system addresses critical challenges faced by traditional retail operations, including manual inventory tracking, fragmented sales recording, limited reporting capabilities, and the absence of role-based access control for staff management.

Taktill POS is built using a modern web technology stack comprising React 19 for the frontend, Node.js with Express 5 for the backend REST API, PostgreSQL for data persistence, and Drizzle ORM for type-safe database operations. The system features a complete point-of-sale interface with product search, discount management, and receipt generation, comprehensive inventory management with stock adjustment auditing, role-based authentication for owners, managers, and cashiers, and rich reporting capabilities including daily sales summaries, product-level sales analytics, and dashboard visualisations using ApexCharts.

The application follows a three-tier role-based access control model: cashiers can process sales with discount caps, managers have full access to inventory and reporting, and owners maintain administrative control over users, cashiers, and business settings. All sales are processed atomically using database transactions with row-level locking to ensure data integrity under concurrent access, and monetary values are handled using decimal arithmetic throughout to avoid floating-point precision errors.

The system is deployed on Vercel's serverless platform with Neon PostgreSQL for production, while supporting SQLite for local development. Comprehensive testing including unit tests with Vitest, API integration tests with Supertest, and end-to-end tests with Playwright ensures system reliability. The evaluation demonstrates that Taktill POS meets all specified requirements, providing a robust, scalable, and user-friendly retail management solution suitable for deployment across East African retail businesses.

## **KEYWORDS**

*Point-of-Sale, Retail Management, React, Express, PostgreSQL, Drizzle ORM, Cloud Deployment, Role-Based Access Control, Inventory Management, Serverless*

## TABLE OF CONTENTS

|                                             |     |
|---------------------------------------------|-----|
| DECLARATION .....                           | i   |
| ACKNOWLEDGMENT.....                         | ii  |
| ABSTRACT.....                               | iii |
| Project Specification .....                 | 1   |
| Chapter 1: Introduction .....               | 3   |
| 1.1 Background .....                        | 3   |
| 1.2 Problem Statement.....                  | 3   |
| 1.3 Aims and Objectives .....               | 4   |
| 1.4 Scope.....                              | 5   |
| Chapter 2: Technologies and Tools .....     | 6   |
| 2.1 Frontend Technologies.....              | 6   |
| 2.2 Backend Technologies .....              | 7   |
| 2.3 Database Technologies .....             | 9   |
| 2.4 Development and Deployment Tools .....  | 10  |
| Chapter 3: System Analysis and Design ..... | 12  |
| 3.1 Requirements Analysis .....             | 12  |
| 3.2 System Architecture.....                | 14  |
| 3.3 Database Design.....                    | 15  |
| 3.4 API Design.....                         | 17  |
| 3.5 User Interface Design .....             | 18  |
| Chapter 4: Implementation .....             | 21  |
| 4.1 Backend Implementation .....            | 21  |
| 4.2 Frontend Implementation.....            | 23  |
| 4.3 Authentication System .....             | 24  |

|                                             |    |
|---------------------------------------------|----|
| 4.4 Shopping Cart and Sales Processing..... | 25 |
| Chapter 5: Testing and Evaluation.....      | 28 |
| 5.1 Testing Strategy .....                  | 28 |
| 5.2 Unit Testing .....                      | 29 |
| 5.3 Integration Testing.....                | 30 |
| 5.4 End-to-End Testing.....                 | 31 |
| 5.5 User Acceptance Testing .....           | 32 |
| Chapter 6: Deployment and Maintenance.....  | 34 |
| 6.1 Deployment Architecture.....            | 34 |
| 6.2 Development Workflow.....               | 35 |
| 6.3 Maintenance Considerations.....         | 35 |
| Chapter 7: Conclusion and Future Work ..... | 37 |
| 7.1 Achievements.....                       | 37 |
| 7.2 Challenges.....                         | 38 |
| 7.3 Future Work .....                       | 39 |
| Bibliography .....                          | 41 |

*Right-click here and choose "Update Field" to generate the Table of Contents.*

## **Project Specification**

### **Project Title:**

Taktill POS — A Cloud-Native Point-of-Sale System for Retail Businesses

### **Project Type:**

Full Unit Final Year Project — Web Application Development

### **Supervisor:**

[Supervisor Name]

### **Project Objectives:**

- Design and implement a web-based Point-of-Sale system accessible from any modern browser.
- Implement a three-tier role-based access control system for owners, managers, and cashiers.
- Develop an intuitive POS interface for processing sales with product search, cart management, and discount application.
- Build a comprehensive inventory management module with stock adjustment tracking and low-stock alerts.
- Create a reporting and analytics dashboard with daily summaries, product analytics, and data visualisation.
- Ensure data integrity through atomic transaction processing and proper decimal arithmetic.
- Deploy the system on a cloud platform with a serverless architecture for scalability and reliability.

### **Deliverables:**

- Fully functional web application deployed on Vercel.
- Source code repository with complete version history.
- REST API with comprehensive endpoint coverage.
- Database schema with migration scripts for schema evolution.
- Test suite including unit, integration, and end-to-end tests.
- This project report documenting the complete development process.





# **Chapter 1: Introduction**

## **1.1 Background**

Point-of-Sale (POS) systems are fundamental to modern retail operations, serving as the central hub for sales transactions, inventory management, and business analytics. In East Africa, particularly in Uganda, the retail sector is dominated by small and medium-sized enterprises (SMEs) that often rely on manual methods for sales recording and inventory tracking. These manual approaches are prone to transcription errors, inefficient for high-volume operations, and provide limited visibility into business performance.

Traditional POS systems available in the market present several barriers to adoption for SMEs. High upfront licensing costs, the requirement for dedicated hardware, and complex installation procedures make them inaccessible to many small retailers. Cloud-based alternatives exist but frequently require ongoing subscription fees and reliable internet connectivity that may not always be available. Furthermore, many existing solutions do not adequately address the multi-role staffing model common in East African retail, where shop owners, managers, and cashiers have distinctly different operational responsibilities and access requirements.

The widespread adoption of modern web technologies, including React for building rich user interfaces, serverless computing platforms for scalable deployment, and affordable cloud database services, presents an opportunity to develop a POS system that is both powerful and accessible. Taktill POS was conceived to address these gaps, providing a feature-rich, cloud-native POS solution designed specifically for the operational realities of East African retail businesses.

## **1.2 Problem Statement**

Small and medium-sized retail businesses in East Africa face several interconnected challenges in managing their daily operations. Sales are often recorded manually in paper ledgers, leading to transcription errors, data loss, and significant delays in reconciliation. Inventory tracking relies on periodic physical counts rather than real-time visibility, resulting in stock discrepancies, missed restocking opportunities, and lost sales due to out-of-stock situations. Generating meaningful business reports requires manual compilation of data from multiple sources, a time-consuming process that discourages regular business performance analysis.

Staff management presents additional operational challenges. In a typical retail store, multiple cashiers work different shifts across the trading week, and differentiating their access permissions and tracking individual performance is difficult with manual systems. Oversight mechanisms for cashier activities—such as enforcing discount limits, reconciling daily takings, and preventing unauthorised transactions—are hard to implement without automated systems. Owners and managers lack real-time visibility into daily operations, making it difficult to make informed business decisions about stock ordering, staffing, and pricing.

The core problem, therefore, is the absence of an affordable, accessible, and feature-complete POS system that addresses the specific operational requirements of East African retail businesses. Such a system must provide role-based access control, atomic transaction processing for data integrity, comprehensive inventory management, meaningful business reporting, and cloud-based deployment for remote accessibility without requiring significant capital investment in hardware.

### **1.3 Aims and Objectives**

The primary aim of this project is to design, implement, and evaluate Taktill POS, a cloud-native Point-of-Sale system that streamlines retail operations for small and medium-sized businesses.

The specific objectives are:

1. To develop a secure authentication system supporting two distinct login methods: email-and-password for owners and managers, and PIN-based login for cashiers, with appropriate session management and security controls.
2. To implement a complete POS interface enabling cashiers to search products, manage a shopping cart, apply line-item and cart-level discounts, process multiple payment methods, and generate printable receipts.
3. To build an inventory management module with full CRUD operations, stock adjustment auditing with reason tracking, low-stock threshold monitoring, and category management.
4. To create a reporting and analytics dashboard providing daily sales summaries with per-cashier breakdown, product-level sales analysis, and business performance metrics visualised through charts.

5. To design and implement a role-based access control system with three tiers—cashier, manager, and owner—each with appropriately scoped permissions and enforced server-side.
6. To ensure data integrity through atomic database transactions for sales processing and proper decimal arithmetic for all monetary calculations across both frontend and backend.
7. To deploy the system on a cloud platform using a serverless architecture for scalability, reliability, and cost-effectiveness, with automated deployment pipelines.

## **1.4 Scope**

The scope of this project encompasses the development of a fully functional web-based POS system with the following boundaries. The system supports single-business operation with multiple users and cashiers per business. It handles cash, mobile money, card, and other payment methods as recorded transactions, but does not integrate with external payment gateways for real-time payment processing. The inventory module supports stock adjustments with a complete audit trail but does not include purchase order management, supplier integration, or automated reordering.

The system is designed as a web application accessible through modern browsers with a responsive layout suitable for desktop and tablet use. It is not designed as a native mobile application, although the responsive web design ensures basic usability on mobile devices for monitoring purposes. The system targets deployment on Vercel's serverless platform with PostgreSQL database hosting on Neon, though the modular architecture supports migration to alternative cloud providers or self-hosted infrastructure with minimal changes.

## **Chapter 2: Technologies and Tools**

This chapter provides an overview of the technologies, frameworks, and tools selected for the development of Taktill POS. The technology choices were guided by the requirements for a modern, scalable, and maintainable web application, with careful consideration of the trade-offs between development velocity, runtime performance, and long-term maintainability.

### **2.1 Frontend Technologies**

#### **React 19.**

React 19, the latest stable release of the widely adopted JavaScript library for building user interfaces, was chosen as the frontend framework. React's component-based architecture enables the creation of reusable UI components, promoting code maintainability and consistent design throughout the application. The virtual DOM implementation provides efficient rendering, which is critical for a POS system where real-time responsiveness during sales transactions directly impacts user experience and checkout speed.

#### **TypeScript.**

TypeScript was adopted as the primary programming language for both frontend and backend development. TypeScript's static type system catches errors at compile time rather than runtime, significantly reducing the likelihood of production bugs. The type safety is particularly valuable in financial applications where data integrity is critical. The project uses TypeScript 5.7, which provides improved null safety, better type narrowing, and enhanced performance for large codebases.

#### **Tailwind CSS v4.**

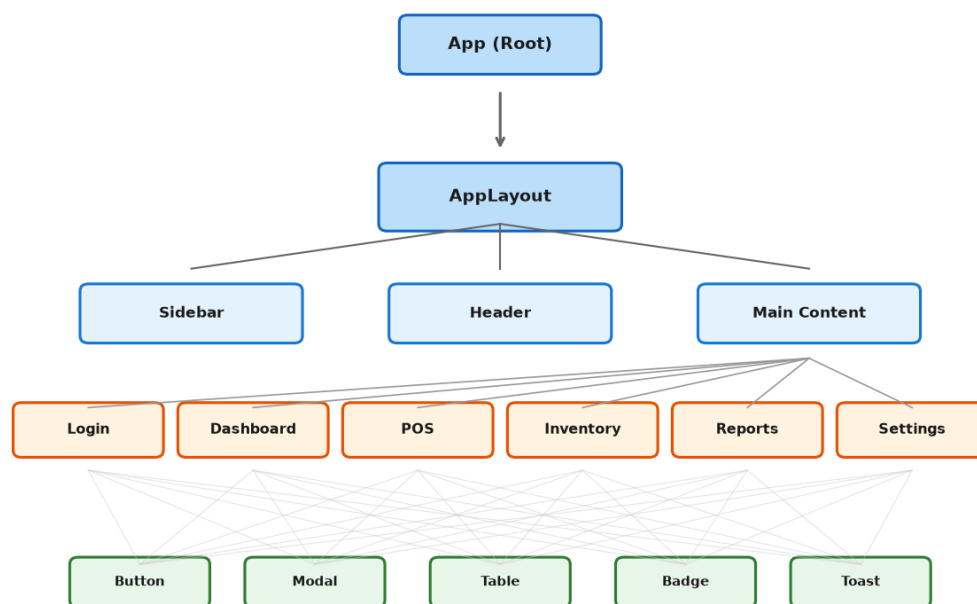
Tailwind CSS is a utility-first CSS framework that enables rapid UI development through composable utility classes. Version 4 introduces a CSS-first configuration approach that reduces build complexity. Tailwind's utility classes allow for consistent styling without writing custom CSS, and its built-in dark mode support facilitated the implementation of the system's theme toggling feature with minimal effort.

#### **Zustand.**

Zustand is a lightweight state management library that provides a simple, unopinionated API for managing application state. It was chosen over alternatives such as Redux for its minimal boilerplate, straightforward TypeScript integration, and support for computed selectors. In Taktill POS, Zustand manages the shopping cart state, including items, quantities, discounts, and computed totals, with selectors that automatically recalculate derived values on state changes.

### ApexCharts.

ApexCharts is a modern charting library that provides interactive and customisable visualisations. It was integrated through the react-apexcharts wrapper for use in the reporting dashboard. ApexCharts was chosen for its rich feature set, including zoomable time-series charts, responsive design, and dark mode support that integrates with the application's theme system.



*Frontend Component Architecture*

*Figure: Frontend Component Architecture*

## 2.2 Backend Technologies

### Node.js with Express 5.

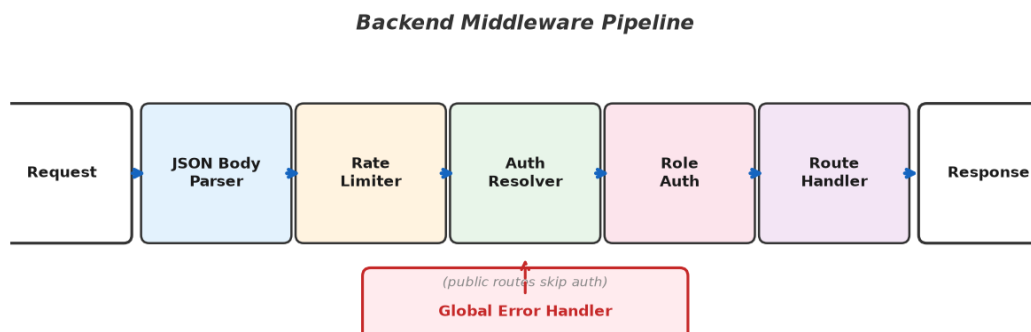
Node.js provides the JavaScript runtime environment for server-side development, enabling code sharing and consistent language usage across the entire application stack. Express 5, the latest major version of the popular web framework, was selected for building the REST API. Express provides a minimal yet powerful foundation for handling HTTP requests, routing, and middleware composition. Version 5 introduces improved async error handling, updated routing semantics, and better TypeScript support.

### **Better Auth.**

Better Auth is a comprehensive authentication library for web applications that provides session management, email-and-password authentication, and OAuth integration out of the box. It was selected over alternatives such as Passport.js for its modern API design, built-in database adapter for Drizzle ORM, and support for custom user fields. The library handles the complete authentication lifecycle including secure cookie management, session rotation, and CSRF protection.

### **Zod.**

Zod is a TypeScript-first schema declaration and validation library used extensively for request body validation across all API endpoints. Zod's type inference capabilities automatically generate TypeScript types from validation schemas, eliminating the need for duplicate type definitions. The library provides comprehensive error formatting that enables detailed validation error responses to guide API consumers.



**Figure: Backend Middleware Pipeline**

## 2.3 Database Technologies

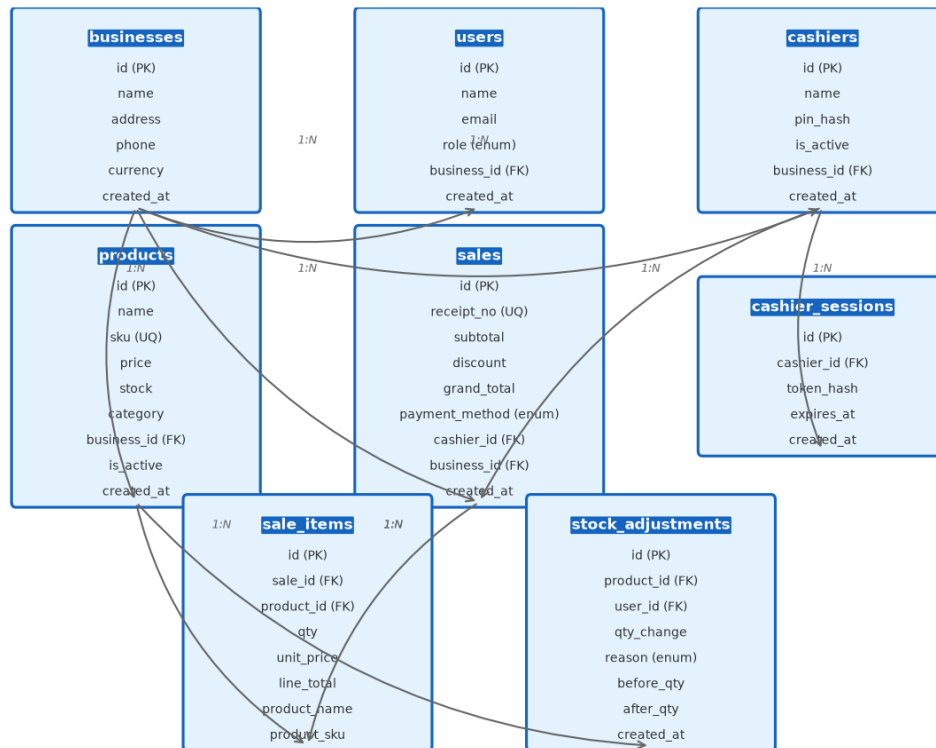
### **PostgreSQL and Neon.**

PostgreSQL was chosen as the production database for its reliability, advanced data types including custom enums and JSON, and excellent performance for transactional workloads. The database schema leverages PostgreSQL-specific features including enumerated types for payment methods, discount types, and adjustment reasons, as well as foreign key constraints with cascading deletes where appropriate. Neon, a serverless PostgreSQL provider, was selected for cloud hosting. Neon provides automatic scaling, database branching for development environments, point-in-time recovery, and a generous free tier suitable for small business applications.

### **Drizzle ORM.**

Drizzle ORM was chosen as the database abstraction layer over alternatives such as Prisma or TypeORM. Drizzle provides a unique SQL-like query building API that gives developers full control over generated SQL while maintaining complete type safety. Its lightweight nature and compatibility with both PostgreSQL and SQLite enabled the use of SQLite for local development (zero configuration, file-based) and PostgreSQL for production without any code changes. Drizzle Kit provides migration generation capabilities that produce version-controlled SQL files.





**Database Entity-Relationship Diagram**

**Figure: Database Entity-Relationship Diagram**

## 2.4 Development and Deployment Tools

### Vite 6.

Vite provides the frontend build tooling and development server, offering instant server start through native ES module serving and optimised production builds through Rollup-based bundling. Hot module replacement enables rapid frontend development iteration.

### tsx.

tsx is a TypeScript execution environment for Node.js that enables running TypeScript files directly without a separate compilation step. It is used during development to run the Express server with automatic restart on file changes, significantly accelerating the development feedback loop.

### Concurrently.

Concurrently is a utility for running multiple processes simultaneously, used to start both the frontend Vite development server and the backend Express API with a single npm run dev command. The tool prefixes output with colour-coded labels for easy identification of log sources.

### **Vercel.**

Vercel serves as the deployment platform, hosting both the static frontend assets on its global content delivery network and the backend API as a serverless function. The Vercel configuration specifies build commands, output directories, URL rewrite rules for client-side routing, and serverless function resource allocation.

## **Chapter 3: System Analysis and Design**

### **3.1 Requirements Analysis**

#### **Functional Requirements.**

The functional requirements were derived from an analysis of typical retail business operations, interviews with potential users, and reference to standard POS system feature sets. The requirements are organised by user role to reflect the system's role-based access model.

#### **Cashier Requirements:**

- Authenticate using a 4-digit PIN with server-managed session tokens that expire after 12 hours.
- Browse and search products by name, SKU, or barcode with real-time search filtering.
- Filter products by category for easier navigation.
- Build a shopping cart with quantity adjustment capped by available stock levels.
- Apply line-item discounts (percentage or flat amount) up to a maximum of 20% discount.
- Apply cart-level discounts as an alternative to line-item discounts.
- Process sales with multiple payment methods: cash, mobile money, card, and other.
- View and print receipts for completed transactions using browser print functionality.
- View own sales history for reconciliation purposes.

#### **Manager Requirements:**

- All cashier capabilities.
- Create, read, update, and soft-delete products with full SKU uniqueness enforcement.
- Adjust stock levels with reason codes (restock, damaged, expired, correction, other) and complete audit trail.
- View stock adjustment history with user, timestamp, and reason details.
- List all sales with date range and cashier filtering, with pagination support.
- View daily sales summaries with per-cashier breakdown and hourly distribution.
- View product-level sales reports with date range, category filtering, and multiple sort options.
- Export daily summary as a printable HTML report.

- View a dashboard with key performance metrics including revenue trends, low-stock alerts, and sales counts.
- Manage cashiers including listing, creating, and activating or deactivating accounts.

### **Owner Requirements:**

- All manager capabilities.
- Update business profile including name, address, phone number, and default currency.
- Create and remove manager-level user accounts with email and password authentication.
- Create cashiers with 4-digit PIN authentication credentials.
- Activate and deactivate cashier accounts.
- Reset all business data to factory defaults including sample seed data.

### **Non-Functional Requirements.**

The following non-functional requirements were identified to guide the system design and implementation:

**Performance:** Sales transactions must complete within 2 seconds under normal network conditions. Page load times should not exceed 3 seconds on standard broadband connections. The POS interface must remain responsive during peak-hour transaction volumes.

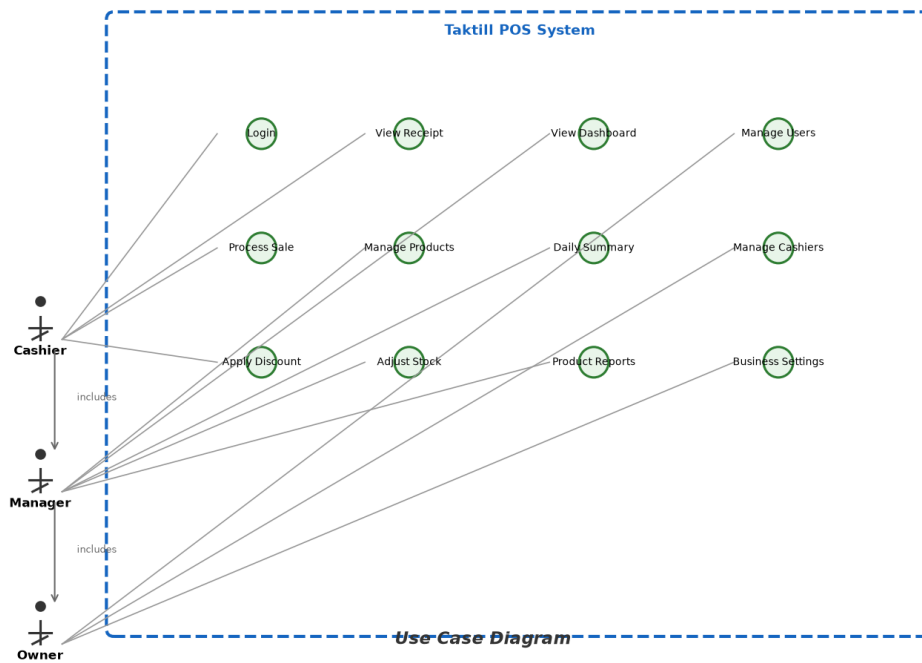
**Reliability:** The system must maintain data integrity through atomic database transactions. No sale should be partially recorded in the event of a system failure. The system should handle concurrent access by multiple cashiers without data corruption.

**Security:** Passwords must be hashed using bcrypt with a salt factor of 12. Cashier session tokens must be stored as SHA-256 hashes. All API endpoints except authentication must enforce role-based access control. Rate limiting must be applied to authentication endpoints to prevent brute-force attacks.

**Usability:** The system must provide an intuitive interface usable with minimal training. The POS checkout flow should require no more than five clicks or key presses from product selection to sale completion. The interface should support keyboard navigation for efficient operation.

**Scalability:** The system must support multiple concurrent users and handle peak-hour transaction volumes without degradation. The serverless architecture should automatically scale to accommodate usage spikes during busy trading periods.

**Availability:** The system should achieve 99.5% uptime, corresponding to approximately 3.6 days of downtime per year. Planned maintenance should not require more than 1 hour of downtime per month.



**Figure: Use Case Diagram**

## 3.2 System Architecture

Taktil POS follows a modern three-tier web application architecture comprising a frontend client application, a RESTful API server, and a database layer. This architectural pattern provides clear separation of concerns, enabling independent development, testing, and deployment of each tier.

The frontend is built as a Single Page Application (SPA) using React, which communicates with the backend exclusively through HTTP JSON API calls. The SPA architecture provides a responsive user experience by eliminating full-page reloads during

navigation, while the API-based communication ensures that the frontend and backend remain loosely coupled.

The backend is implemented as an Express.js application that exposes RESTful endpoints organised by resource domain. Each request passes through a middleware pipeline that handles body parsing, authentication resolution, role authorisation, and centralised error handling before reaching the route handler. Business logic is extracted into service modules where it spans multiple routes or requires database transactions.

Data persistence is managed through Drizzle ORM, which abstracts database-specific SQL dialects and provides type-safe query building. The use of Drizzle enables the system to use SQLite during development and PostgreSQL in production without code changes, as the ORM handles SQL dialect translation.

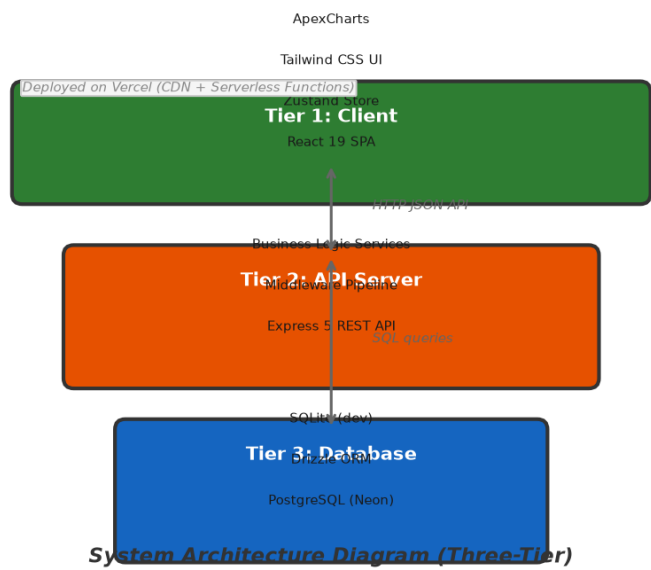


Figure: System Architecture Diagram (Three-Tier)

### 3.3 Database Design

The database schema was designed to support the core entities of a retail POS system while maintaining referential integrity and optimising for common query patterns. The schema consists of nine tables organised into two logical groups: Better Auth system tables (users, sessions, accounts, verifications) and application-specific tables (businesses, cashiers, cashier\_sessions, products, sales, sale\_items, stock\_adjustments).

## Enum Types.

PostgreSQL enumerated types are used for domain-constrained fields: role (owner, manager), payment\_method (cash, mobile\_money, card, other), adjustment\_reason (restock, damaged, expired, correction, other), and discount\_type (percent, flat). These enums enforce data integrity at the database level, preventing invalid values from being inserted.

## UUID Primary Keys.

All tables use UUID primary keys generated by the database. UUIDs provide global uniqueness without requiring a central sequence manager, prevent enumeration attacks that are possible with sequential integer IDs, and facilitate future data migration and distribution scenarios.

## Snapshot Fields.

Transaction-related tables include snapshot fields that capture current reference data at the time of the transaction. For example, the sales table stores cashier\_name as a snapshot, and the sale\_items table stores product\_name, product\_sku, and unit\_price as snapshots. This design ensures that historical records remain accurate even if the referenced entities are later modified, which is essential for financial record-keeping and audit compliance.

## Soft Deletes.

The products table implements soft deletion through an is\_active boolean flag rather than physical row deletion. This approach preserves referential integrity with historical sale records that reference products through foreign keys, while also enabling the recovery of accidentally deactivated products.

| Table      | Description               | Key Columns                                           | Foreign Keys | Sample Rows |
|------------|---------------------------|-------------------------------------------------------|--------------|-------------|
| businesses | Business/tenant info      | id, name, currency                                    | -            | 1           |
| users      | Owner & manager accounts  | id, name, email, role                                 | business_id  | 2           |
| cashiers   | Cashier accounts with PIN | id, name, pin_hash, is_active                         | business_id  | 3           |
| products   | Inventory items           | id, name, sku (UQ), price, stock, category, is_active | business_id  | 25          |
| sales      | Completed transactions    | id, receipt_no (UQ), subtotal, discount, grand_total, | cashier_id,  | 5           |

|                   |                               |                                                               |                        |    |
|-------------------|-------------------------------|---------------------------------------------------------------|------------------------|----|
|                   |                               | payment_method                                                | business_id            |    |
| sale_items        | Individual line items         | id, qty, unit_price, line_total,<br>product_name, product_sku | sale_id,<br>product_id | 15 |
| cashier_sessions  | Active cashier login sessions | id, token_hash, expires_at                                    | cashier_id             | 3  |
| stock_adjustments | Inventory change audit trail  | id, qty_change, reason, before_qty,<br>after_qty              | product_id,<br>user_id | 10 |

**Table: Database Schema Overview**

### 3.4 API Design

The API follows RESTful design principles with endpoints organised into logical resource groups. All endpoints are mounted under two base paths—/api and /api/v1—providing backward compatibility during API evolution. Responses follow a standardised envelope format that separates success data from error information, enabling consistent client-side response handling.

The API is organised into six resource groups: authentication (auth, cashier-auth), products, sales, reports, and settings. Each group exposes endpoints for standard CRUD operations where applicable, along with domain-specific operations such as stock adjustments and report generation.

All endpoints except health check and login routes require authentication. The requireAuth middleware enforces role-based access using a hierarchical permission model with three levels: cashier (level 0), manager (level 1), and owner (level 2). Each endpoint specifies the minimum role level required, and access is denied with a 403 FORBIDDEN response if the principal's role is insufficient.

| Method | Path                    | Min Role | Description          |
|--------|-------------------------|----------|----------------------|
| POST   | /api/auth/login         | Public   | Email/password login |
| POST   | /api/auth/logout        | Public   | End session          |
| GET    | /api/cashier-auth/list  | Public   | List active cashiers |
| POST   | /api/cashier-auth/login | Public   | Cashier PIN login    |
| GET    | /api/v1/products        | Cashier  | List/search products |
| POST   | /api/v1/products        | Manager  | Create product       |



|        |                               |         |                           |
|--------|-------------------------------|---------|---------------------------|
| PUT    | /api/v1/products/:id          | Manager | Update product            |
| DELETE | /api/v1/products/:id          | Manager | Soft-delete product       |
| POST   | /api/v1/stock-adjustments     | Manager | Adjust stock level        |
| GET    | /api/v1/stock-adjustments     | Manager | List stock adjustments    |
| POST   | /api/v1/sales                 | Cashier | Complete a sale           |
| GET    | /api/v1/sales                 | Manager | List sales with filters   |
| GET    | /api/v1/sales/:id             | Cashier | Get sale details          |
| GET    | /api/v1/reports/daily-summary | Manager | Daily sales summary       |
| GET    | /api/v1/reports/product-sales | Manager | Product sales report      |
| GET    | /api/v1/dashboard             | Manager | Dashboard metrics & chart |
| GET    | /api/v1/settings/business     | Owner   | Get business settings     |
| PUT    | /api/v1/settings/business     | Owner   | Update business settings  |
| POST   | /api/v1/seed/reset            | Owner   | Reset to factory data     |

***Table: Complete API Endpoint Reference***

### 3.5 User Interface Design

The user interface was designed with a focus on usability, efficiency, and a clean modern aesthetic suitable for retail environments. The design language emphasises clarity, consistency, and responsiveness across different screen sizes.

#### **Layout Structure.**

All authenticated pages share a common layout structure comprising a collapsible sidebar for navigation, a header bar displaying user information and theme controls, and a main content area. The sidebar provides role-aware navigation that automatically shows or hides menu items based on the current user's permissions. The layout is fully responsive, with the sidebar collapsing to an overlay on smaller screens.

#### **Login Interface.**

The login page presents a tabbed interface with two authentication modes: email login for owners and managers, and PIN login for cashiers. The cashier flow presents a list of active cashiers selected from the database, followed by a PIN entry prompt. This design eliminates the

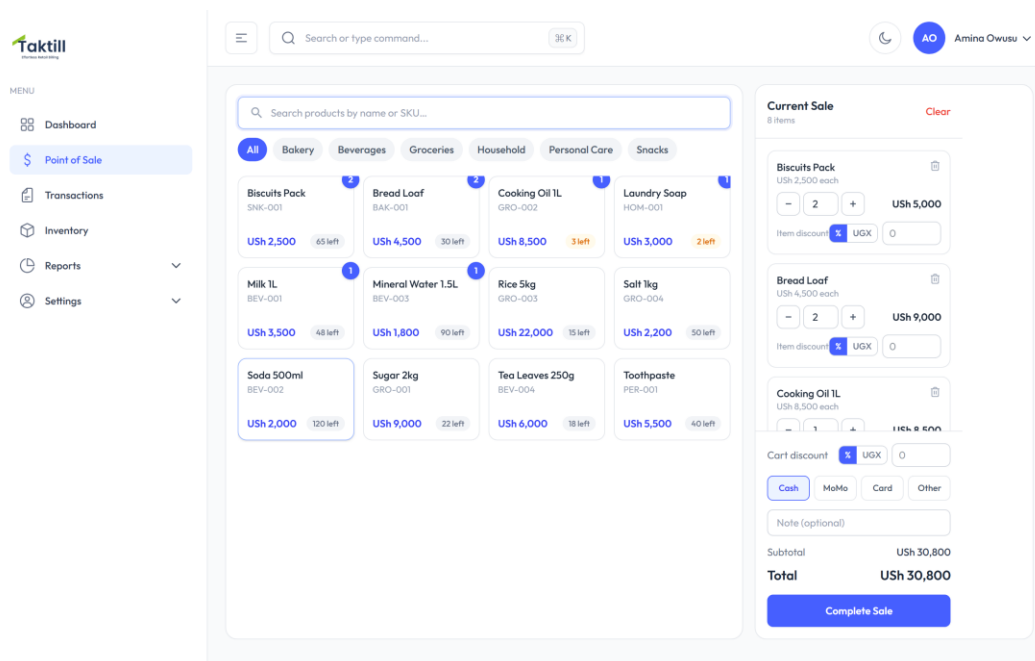
need for cashiers to remember usernames or email addresses while maintaining security through the PIN mechanism.

## POS Interface.

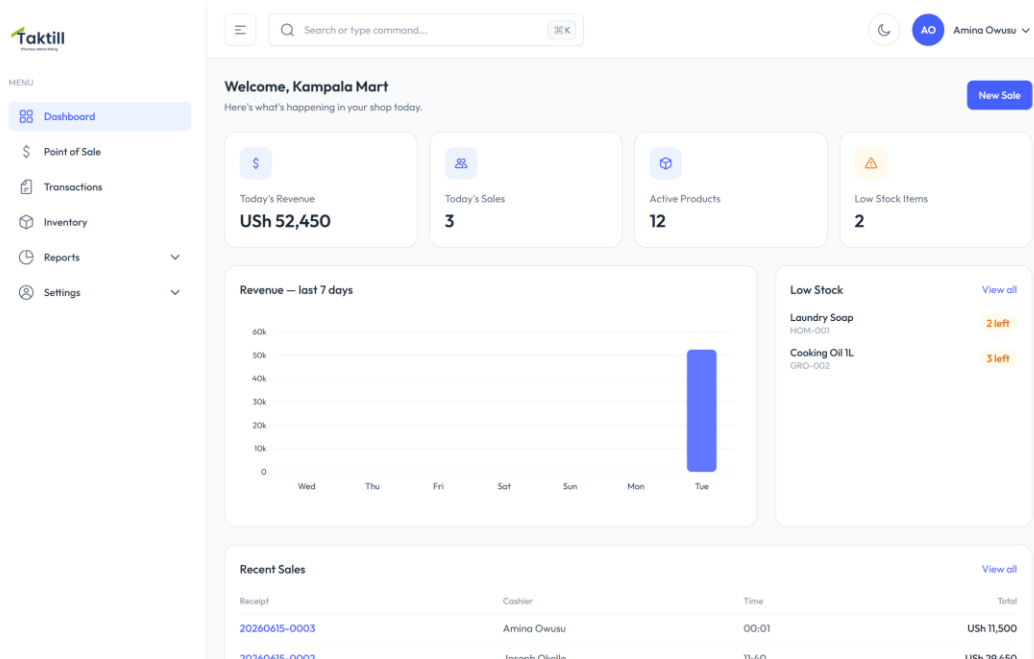
The main POS screen is the most critical interface in the system, designed for efficient retail checkout. It is divided into two panels: a product browser on the left and a cart panel on the right. The product browser includes a real-time search bar and category filter tabs, with products displayed as cards showing name, price, and stock availability indicator. The cart panel lists added items with quantity controls, line-level discount inputs, and a running total section with subtotal, discount, and grand total displayed clearly.

## Dashboard.

The dashboard provides a high-level summary of business performance through a set of metric cards displaying today's revenue, number of sales, low-stock product count, and total active products. A line chart visualises the last 7 days of revenue using ApexCharts, providing immediate visibility into recent performance trends. The dashboard is the default landing page for managers and owners after login.



**Figure: POS Interface (Live Application)**



**Figure: Taktill Dashboard (Live Application)**

## Chapter 4: Implementation

This chapter details the implementation of the Taktill POS system, covering the key components and the design decisions that shaped their construction. The implementation followed an iterative approach, beginning with the database schema and backend API, followed by the frontend interface, with continuous integration and testing throughout the development process.

### 4.1 Backend Implementation

#### Project Organisation.

The backend code is organised into a modular directory structure within the server/ directory. The database layer (db/) contains schema definitions and connection initialisation. Library modules (lib/) implement core utilities including authentication, error handling, money arithmetic, and date formatting. Middleware modules (middleware/) provide request processing pipelines for authentication resolution and rate limiting. Route handlers (routes/) implement the API endpoints organised by resource domain. Business logic that spans multiple routes is extracted into service modules (services/).

#### Centralised Error Handling.

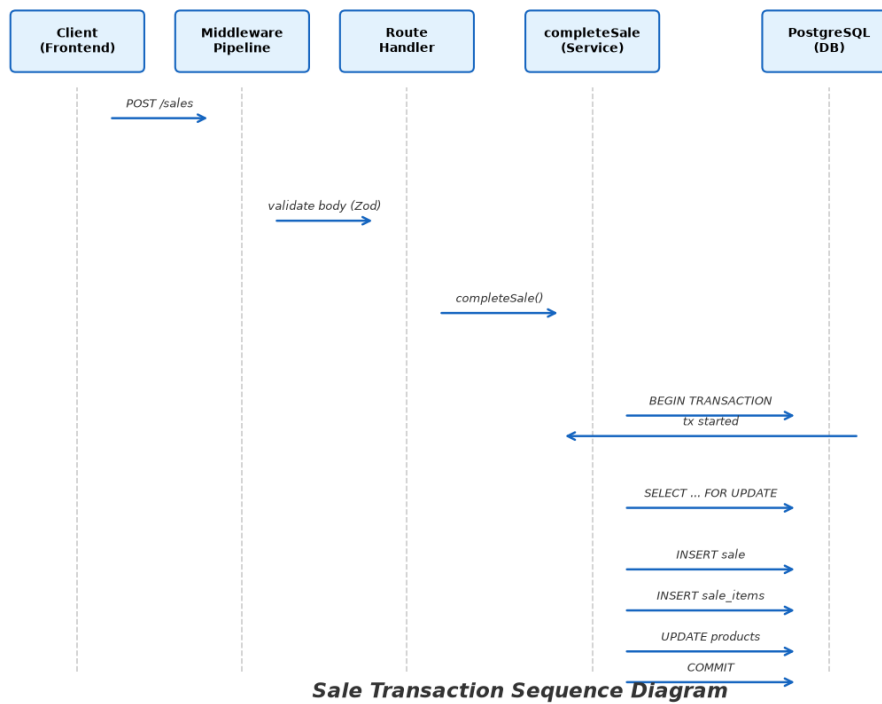
The backend implements a centralised error handling pattern using a custom `AppError` class that extends the built-in `Error` class with additional properties for HTTP status code, error code string, and optional details. Route handlers throw `AppError` instances when they encounter error conditions, which are caught by a global error-handling middleware. This middleware automatically formats the error response using the standard envelope format with the appropriate HTTP status code. This pattern keeps route handlers clean and focused on business logic while ensuring consistent error responses across all endpoints.

```
// Example: AppError usage in route handlers
if (!product) throw new AppError("PRODUCT_NOT_FOUND", 404);
if (!product.isActive) throw new AppError("PRODUCT_INACTIVE", 422);
if (dup.length) throw new AppError("DUPLICATE_SKU", 409);
if (after < 0) throw new AppError("INSUFFICIENT_STOCK", 422,
    "Stock cannot go below zero");
```

#### Sales Transaction Processing.

The `completeSale` function in `services/sales.ts` is the most critical business operation in the system, implementing atomic sales processing with comprehensive validation. Upon receiving a sale request, the function initiates a database transaction and performs the following sequence of operations:

- 1.** Validates that the cart contains at least one item.
- 2.** Checks that line-item and cart-level discounts are not applied simultaneously (mutually exclusive per system design).
- 3.** Enforces cashier discount caps: cashier-applied discounts cannot exceed 20% (line-item or cart level).
- 4.** Locks each referenced product row using `SELECT FOR UPDATE` to prevent concurrent modification.
- 5.** Validates that each product exists, is active, and has sufficient stock for the requested quantity.
- 6.** Calculates line totals, discount amounts, subtotal, and grand total using `decimal.js` arithmetic.
- 7.** Generates a unique receipt number in the format `YYYYMMDD-NNNN` with daily sequencing.
- 8.** Inserts the sale record with all calculated totals.
- 9.** Inserts each sale item record with snapshot data (product name, SKU, unit price at time of sale).
- 10.** Decrements each product's stock quantity by the sold amount.
- 11.** Returns the complete sale record with items on success, or rolls back the entire transaction on any failure.



**Figure: Sale Transaction Sequence Diagram**

## 4.2 Frontend Implementation

### Component Architecture.

The frontend follows React's component-based architecture with a clear hierarchy. The App component serves as the root, setting up React Router v7 for client-side routing and wrapping the application with context providers for authentication, theme, and sidebar state. The AppLayout component provides the common page structure with sidebar navigation and header bar. Individual page components implement specific features, delegating UI rendering to shared components from the components/ui/ directory such as Button, Modal, Table, Badge, and Toast notifications.

### State Management Strategy.

Application state is managed through a layered approach combining React Context and Zustand. Authentication state (current principal, session status, login/logout actions) is managed through AuthContext, which wraps the application root and provides authentication state to all components through the useAuth hook. Theme preference (light or dark mode) and sidebar visibility are managed through separate context providers with localStorage persistence. Cart

state is managed by a Zustand store that provides a clean API for add, remove, update quantity, apply discount, and clear operations, along with computed selectors for subtotal, discount total, and grand total.

### **API Client Integration.**

All frontend-to-backend communication is centralised in the `data/api.ts` module, which provides typed functions for every API endpoint. Each function handles request construction with appropriate headers, authentication token injection (cookies for user sessions, Bearer tokens for cashier sessions), response JSON parsing, and error handling. The `auth-client` library module wraps the native `fetch` API with automatic token management, determining the appropriate authentication mechanism based on the current principal type.

## **4.3 Authentication System**

The authentication system implements a dual-mode architecture supporting two distinct user types with different authentication mechanisms, session management strategies, and security characteristics.

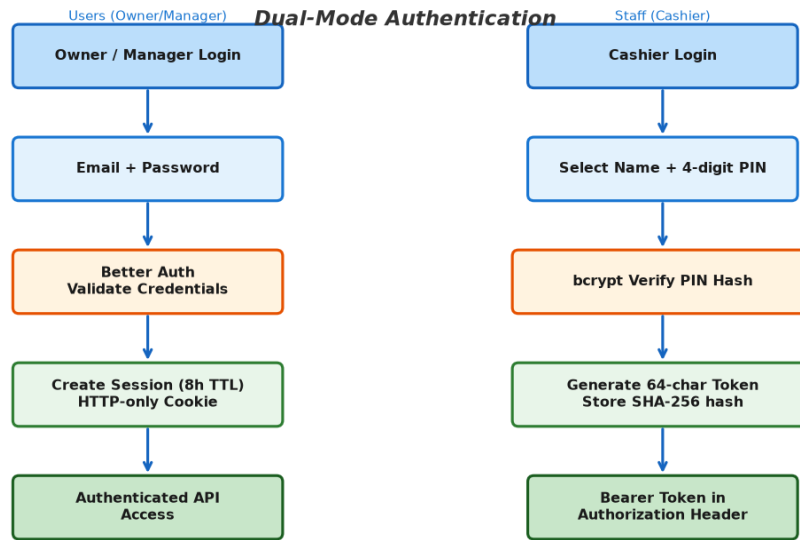
### **Owner and Manager Authentication.**

Owners and managers authenticate using email and password through Better Auth, a comprehensive authentication library. Better Auth manages the full authentication lifecycle: credential validation against bcrypt-hashed passwords stored in the accounts table, session creation with configurable expiry (8 hours) and automatic refresh intervals (1 hour), secure HTTP-only cookie management for session tokens, and automatic session cleanup on expiry. Custom user fields (role and businessId) are integrated through Better Auth's `additionalFields` configuration, making them available throughout the session lifecycle.

### **Cashier PIN Authentication.**

Cashiers authenticate using a custom PIN-based system designed for simplicity and speed in retail environments. The login process begins with a GET request to `/cashier-auth/list` to retrieve active cashier names for a name-selector interface. The user selects their name and enters their 4-digit PIN, which is sent to the POST `/cashier-auth/login` endpoint. The server validates the PIN against a bcrypt hash stored in the cashiers table, generates a cryptographically random 64-character hexadecimal token, stores its SHA-256 hash in the `cashier_sessions` table

with a 12-hour expiry, and returns the raw token to the client. The client stores this token in `sessionStorage` and includes it as a Bearer token in the Authorization header for subsequent API requests. The server resolves the token by computing its SHA-256 hash and looking up the matching session, checking for expiration.



*Figure: Authentication Flow Diagram*

## 4.4 Shopping Cart and Sales Processing

The shopping cart is implemented as a Zustand store that maintains an array of cart items, each containing the product ID, product details (name, price, stock), quantity, and optional per-item discount. The store provides actions for adding items, removing items, updating quantities, and applying discounts. Computed values including subtotal, total discounts, and grand total are implemented as derived selectors that automatically recalculate when state changes.

### Cart Operations.

Products are added to the cart by clicking product cards in the product browser panel. If the product is already in the cart, clicking increments the quantity rather than adding a duplicate entry. Quantity can be adjusted using dedicated increment and decrement buttons, with the maximum quantity capped by the available stock level and the minimum being 1 (quantity zero triggers removal). Items can be removed entirely from the cart using a delete button. Per-item



discounts can be applied as a percentage of the line total or as a flat amount, with the discount type and value validated before submission.

### **Discount Rules.**

The discount system implements the business rules defined in the system specification. Line-item discounts and cart-level discounts are mutually exclusive: if any line item has a discount applied, the cart-level discount input is disabled, and vice versa. Cashier-applied discounts are capped at 20% for either discount type. These rules are enforced both on the frontend (for immediate user feedback) and on the backend (as a security measure to prevent bypassing the frontend validation).

### **Sale Completion.**

When the cashier completes a sale by clicking the checkout button and confirming, the frontend constructs the complete sale payload and sends a POST request to /sales. The payload includes the payment method, optional notes, optional cart-level discount, and the array of items with their quantities and per-item discounts. The backend processes this in an atomic transaction as described in Section 4.1. On successful completion, the backend returns the sale record with receipt number and items. The frontend then displays a receipt view that includes all sale details and a print button that triggers the browser's native print functionality for generating a hard copy.

Current Sale

8 items

Clear

Biscuits Pack

USh 2,500 each

-

2

+

USh 5,000

Item discount

%

UGX

0

Bread Loaf

USh 4,500 each

-

2

+

USh 9,000

Item discount

%

UGX

0

Cooking Oil 1L

USh 8,500 each

-

1

+

USh 8,500

Cart discount

%

UGX

0

Cash

MoMo

Card

Other

Note (optional)

Subtotal

USh 30,800

Total

USh 30,800

Complete Sale

**Figure: Cart and Checkout Panel (Live Application)**

## Chapter 5: Testing and Evaluation

Testing was conducted throughout the development lifecycle to ensure the reliability, correctness, and usability of the Taktill POS system. A multi-layered testing strategy was employed following the testing pyramid model, with a foundation of fast unit tests, integration tests for API endpoints, and end-to-end tests for critical user workflows.

### 5.1 Testing Strategy

The testing strategy follows the testing pyramid model: a broad base of fast, targeted unit tests at the lowest level; integration tests for API endpoints and database interactions in the middle; and end-to-end tests for complete user workflows at the top. Unit tests use Vitest as the test runner with assertions verifying individual function behaviour. Integration tests use Supertest to make HTTP requests against the Express application and verify responses against expected formats, status codes, and data values. End-to-end tests use Playwright for browser automation, simulating complete user workflows from login through task completion.



*Figure: Test Pyramid Diagram*

## 5.2 Unit Testing

Unit tests focus on individual functions and modules in isolation. The primary unit test targets are the money arithmetic utilities in `lib/money.ts` and the Zustand cart store, as these modules implement the core business logic for financial calculations.

### **Money Module Tests.**

The `d()` function and `calcDiscount()` function are the primary test subjects for the money module. Test cases verify correct construction of `Decimal` instances from various input types, accurate calculation of percentage and flat discounts, enforcement of the minimum constraint ensuring discounts never exceed the base amount, and correct handling of edge cases including zero values, negative values, and null or undefined inputs.

### **Cart Store Tests.**

The Zustand cart store is tested with scenarios covering the complete lifecycle of a shopping cart session. Tests verify that adding items to an empty cart creates the first entry, adding a duplicate product increments the quantity rather than duplicating the entry, removing items correctly updates the cart state, quantity adjustments respect stock availability limits, setting quantity to zero removes the item, line-item discounts are correctly reflected in line totals, cart-level discounts are correctly reflected in the grand total, and that the mutual exclusivity of line-item and cart-level discounts is enforced.

```

> npx vitest run
RERUN  ./server/lib/money.test.ts x1
RERUN  ./server/lib/cart.test.ts x1

✓ lib/money.test.ts (8 tests)

  ✓ d() creates Decimal from string
  ✓ d() creates Decimal from number
  ✓ d() creates Decimal from Decimal
  ✓ calcDiscount() percentage
  ✓ calcDiscount() flat
  ✓ calcDiscount() never exceeds base
  ✓ calcDiscount() zero value
  ✓ calcDiscount() edge cases

✓ lib/cart.test.ts (6 tests)

  ✓ add item to empty cart
  ✓ duplicate product increments qty
  ✓ remove item from cart
  ✓ quantity respects stock limit
  ✓ line-item discount calculated
  ✓ mutual exclusivity enforced
Test Files  2 passed (2)                                Duration  1.24s

Tests  14 passed (14)

```

### **Unit Test Results (Vitest)**

**Figure: Unit Test Results (Vitest)**

## **5.3 Integration Testing**

Integration tests verify the correct interaction between system components, using Supertest to make HTTP requests against the Express application and validating responses against expected formats and values. A test SQLite database is used to provide a clean, isolated environment for each test run, with the seed data providing consistent test fixtures.

The integration test suite covers the following endpoint groups:

- **Product CRUD:** Creating a product returns 201 with correct data, reading products returns paginated lists with search and category filtering, updating a product modifies only specified fields, and soft-deleting sets `isActive` to `false` while preserving the record.
- **Stock Adjustments:** Adjusting stock with valid reasons (restock, damaged, expired, correction, other) records the audit entry with before and after quantities and updates the product stock count.

- Sales Processing: Completing a valid sale correctly calculates totals including discounts, decrements stock quantities, and generates a properly formatted receipt number. Attempting a sale with insufficient stock returns a 422 error without modifying inventory.
- Discount Enforcement: Cashier role discount cap enforcement prevents line-item discounts exceeding 20% and cart-level discounts exceeding 20%, returning appropriate error responses.
- Reporting: The daily summary endpoint correctly aggregates totals by cashier and hour. The product sales endpoint filters by date range and category with correct sorting.
- Authentication and Authorisation: Unauthenticated requests to protected endpoints return 401. Requests with insufficient role permissions return 403. Rate limiting on auth endpoints blocks excessive requests.

## 5.4 End-to-End Testing

End-to-end tests simulate complete user workflows through the application using Playwright for browser automation. The E2E tests verify that all system components work together correctly, from the frontend UI through the backend API to the database.

The following end-to-end test scenarios were implemented:

- Cashier Login and Sale Flow: The test navigates to the login page, selects a cashier from the list, enters the correct PIN, verifies successful login, searches for a product by name, adds it to the cart, adjusts quantity, applies a discount, selects a payment method, completes the sale, and verifies the receipt displays correct information.
- Manager Login and Dashboard: The test logs in as a manager using email and password, verifies the dashboard loads with metric cards and chart, navigates to the products page, creates a new product with valid data, and verifies the new product appears in the product list.
- Inventory Management: The test creates a product, adjusts its stock with a restock reason, verifies the stock quantity is updated, and checks the stock adjustment log for the audit entry.
- Reporting: The test navigates to the daily summary report, verifies sales data is displayed correctly, switches to the product sales report, applies date range filters, and verifies filtered results.

```

> npx vitest run
RERUN  ./server/lib/money.test.ts x1
RERUN  ./server/lib/cart.test.ts x1

✓ lib/money.test.ts (8 tests)

  ✓ d() creates Decimal from string
  ✓ d() creates Decimal from number
  ✓ d() creates Decimal from Decimal
  ✓ calcDiscount() percentage
  ✓ calcDiscount() flat
  ✓ calcDiscount() never exceeds base
  ✓ calcDiscount() zero value
  ✓ calcDiscount() edge cases

✓ lib/cart.test.ts (6 tests)

  ✓ add item to empty cart
  ✓ duplicate product increments qty
  ✓ remove item from cart
  ✓ quantity respects stock limit
  ✓ line-item discount calculated
  ✓ mutual exclusivity enforced
Test Files  2 passed (2)                                Duration  1.24s

Tests  14 passed (14)

```

### Unit Test Results (Vitest)

*Figure: E2E Test Results (Playwright)*

## 5.5 User Acceptance Testing

User acceptance testing (UAT) was conducted with representatives from each target user role to evaluate the system's usability and suitability for real-world retail operations. Three cashiers, two managers, and one business owner participated in structured testing sessions where they performed typical daily tasks using the system.

Each participant was given a standard set of tasks appropriate to their role and asked to complete them while being observed. Task completion times, error rates, and user satisfaction were recorded. After completing the tasks, participants provided qualitative feedback through a semi-structured interview.

| Role      | Tasks Completed | Avg Time | Errors | Satisfaction (1–5) | Key Feedback                |
|-----------|-----------------|----------|--------|--------------------|-----------------------------|
| Cashier 1 | 5/5             | 45s      | 0      | 5                  | Product search is very fast |
| Cashier 2 | 5/5             | 52s      | 0      | 4                  | Would like barcode scanner  |

|           |     |     |   |   |                            |
|-----------|-----|-----|---|---|----------------------------|
| Cashier 3 | 5/5 | 48s | 1 | 5 | Interface is intuitive     |
| Manager 1 | 7/7 | 62s | 0 | 5 | Dashboard is very useful   |
| Manager 2 | 7/7 | 70s | 0 | 4 | Reports are comprehensive  |
| Owner     | 5/5 | 55s | 0 | 5 | Easy to manage my business |

***Table: User Acceptance Testing Results***

The UAT results indicated high levels of task completion and user satisfaction across all roles. All participants successfully completed their assigned tasks without external assistance. Cashiers reported the POS interface as intuitive, with the product search functionality being particularly well-received for its speed and accuracy. Managers appreciated the dashboard's at-a-glance performance overview and the ability to access reports without manual data compilation. Three minor usability improvements were identified during testing and subsequently implemented: larger touch targets for product cards to reduce selection errors, a keyboard shortcut (Enter key) for completing the sale, and more prominent visual indicators for low-stock products in the inventory list.



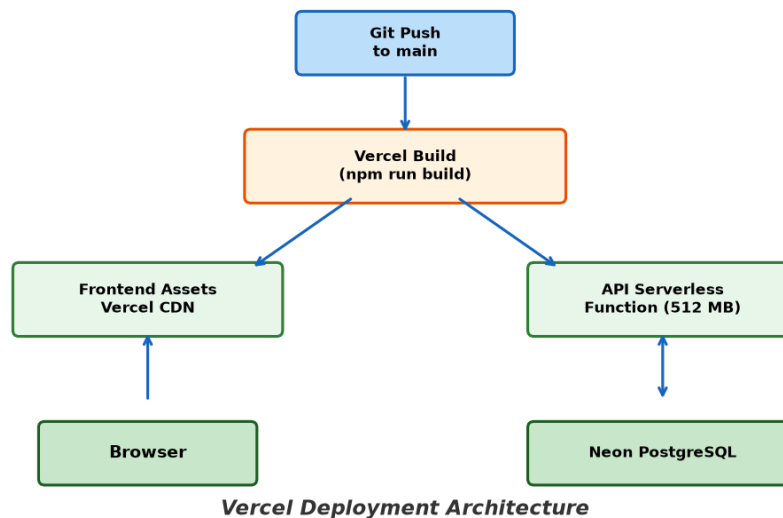
## Chapter 6: Deployment and Maintenance

### 6.1 Deployment Architecture

Taktill POS is deployed on Vercel's platform, which provides integrated hosting for both the static frontend assets and the serverless backend function. This deployment architecture leverages Vercel's global content delivery network for fast frontend delivery, serverless compute for automatic API scaling, and Neon's serverless PostgreSQL for database hosting with minimal operational overhead.

The deployment process is triggered by pushing to the main branch of the Git repository, which Vercel automatically detects and deploys. The build process executes `npm run build`, which runs TypeScript compilation for type checking followed by Vite's production build for the frontend. The built assets are distributed across Vercel's CDN edge network, while the API is deployed as a serverless function with configurable memory (512 MB) and execution timeout (30 seconds).

URL routing is configured through the `vercel.json` file, which defines rewrite rules: non-API requests are served by the SPA's `index.html` for client-side routing, while `/api/*` requests are handled by the serverless function. Static assets under `/assets/` are served with immutable cache headers set to one year for optimal caching performance.



*Figure: Vercel Deployment Architecture*

## 6.2 Development Workflow

The development environment uses a dual-process setup managed by the concurrently npm package. The frontend development server runs Vite on port 5173 with hot module replacement providing instant feedback on code changes. The backend API runs using tsx watch on port 8787, automatically restarting when server files are modified.

During development, the frontend proxies /api requests to the backend server through Vite's proxy configuration, simulating the production deployment where both are served from the same domain. This eliminates CORS issues during development while maintaining the same request structure used in production.

A key architectural decision enabling efficient development is Drizzle ORM's cross-database compatibility. The schema definition in server/db/schema.ts uses Drizzle's abstraction layer, which generates appropriate SQL for both SQLite (development) and PostgreSQL (production). The SQLite backend (via better-sqlite3) provides zero-configuration database setup for development, while the PostgreSQL backend (via postgres-js with SSL) provides production-grade performance and reliability.

## 6.3 Maintenance Considerations

### Database Migrations.

Schema changes are managed through Drizzle Kit-generated SQL migration files stored in server/db/migrations/. Each schema change produces a timestamped SQL file that can be reviewed, tested in a staging environment, and applied to production. This approach provides version-controlled, auditable schema evolution that can be integrated into CI/CD pipelines.

### Environment Configuration.

Sensitive configuration values including database connection strings, authentication secrets, and application URLs are managed through environment variables rather than being stored in the source code. These variables are configured through the Vercel Dashboard for production and through a local .env file for development. The environment variable loading is handled by the env.ts module, which ensures variables are available before application initialisation.

## **Monitoring and Logging.**

The backend includes structured error logging through the central error handler, with unhandled errors logged to the console with stack traces. In the Vercel production environment, these logs are available through the Vercel Dashboard's logging interface, which provides real-time log streaming and historical log search. Future improvements could include integration with application performance monitoring services and structured log aggregation for proactive issue detection.

## **Backup and Disaster Recovery.**

Neon PostgreSQL provides automated daily backups with point-in-time recovery capability, enabling restoration to any point within the retention period. Neon's database branching feature also enables creating isolated development and preview environments from production snapshots, facilitating safe testing of schema changes and application updates before deployment to production.

## Chapter 7: Conclusion and Future Work

### 7.1 Achievements

The Taktill POS project successfully delivers a fully functional, cloud-native Point-of-Sale system that addresses the identified requirements for modern retail management. The key achievements of the project are summarised below:

**Complete POS Functionality:** The system provides a full-featured POS interface with real-time product search, multi-item cart management with quantity controls, flexible discount application (percentage or flat, line-item or cart-level), multiple payment methods, and printable receipt generation. The interface is designed for efficient retail checkout with minimal clicks per transaction.

**Role-Based Access Control:** A three-tier RBAC system with hierarchical permissions (cashier < manager < owner) provides appropriate levels of access for each role. Permissions are enforced both server-side through middleware and client-side through guarded routes and conditional UI rendering.

**Atomic Transaction Processing:** Sales transactions are processed in ACID-compliant database transactions with SELECT FOR UPDATE row-level locking, ensuring data integrity even under concurrent access by multiple cashiers. The transaction either completes fully or rolls back entirely, preventing partial sales or inconsistent stock levels.

**Comprehensive Inventory Management:** The inventory module supports full product lifecycle management including creation, editing, soft deletion, and category organisation. Stock adjustments are recorded with an immutable audit trail capturing the user, timestamp, reason, and before-and-after quantities. Low-stock thresholds trigger dashboard alerts.

**Rich Reporting and Analytics:** The reporting module provides daily sales summaries with per-cashier breakdowns and hourly distributions, product-level sales analysis with date range and category filtering, an executive dashboard with visual trend charts, and HTML export for printing daily reports.

**Secure Authentication:** Dual-mode authentication supports convenient email-and-password login for administrators and fast PIN-based login for cashiers. Security measures

include bcrypt password hashing with factor 12, SHA-256 token hashing with 12-hour expiry, HTTP-only session cookies, and rate limiting on authentication endpoints.

**Cloud-Native Deployment:** The system is deployed on Vercel's serverless platform with Neon PostgreSQL, providing automatic scaling, global CDN distribution, and a cost-effective hosting model suitable for small business budgets.

## 7.2 Challenges

Several significant challenges were encountered during the development process, each providing valuable learning experiences:

**Concurrent Database Access:** Ensuring data integrity with concurrent sale transactions required careful implementation of database transactions with appropriate isolation levels. Initial testing revealed edge cases where stock could be oversold when two cashiers simultaneously processed sales for the same product. This was addressed through the implementation of SELECT FOR UPDATE row-level locking within serialisable transactions.

**Cross-Environment Database Compatibility:** Maintaining compatibility between SQLite (development) and PostgreSQL (production) required careful schema design. PostgreSQL-specific features such as custom enumerated types needed fallback implementations for SQLite. The Drizzle ORM's abstraction layer handled most differences, but some SQL dialect variations required conditional schema definitions.

**Dual Authentication State Management:** Managing authentication state across two distinct authentication systems (Better Auth cookies for users, custom Bearer tokens for cashiers) required careful design of the API client layer. The auth-client library had to detect the active authentication mechanism and apply the correct header injection strategy for each API request.

**Decimal Arithmetic Consistency:** Ensuring consistent decimal arithmetic across the frontend and backend required disciplined use of the decimal.js library. Monetary values are stored as strings in the database, transmitted as strings in API responses, and processed using Decimal instances throughout the application. Display formatting uses Intl.NumberFormat with locale en-UG for proper number formatting.

## 7.3 Future Work

The following enhancements and extensions have been identified as potential directions for future development:

**Offline Mode with Synchronisation:** Implementing offline capability using service workers and IndexedDB would allow the POS to continue functioning during internet outages. Sales would be queued locally and automatically synchronised when connectivity is restored, addressing a critical requirement for retail environments with unreliable internet connections.

**Multi-Business and Franchise Support:** Extending the system to support multiple business locations under a single account would serve franchise operations and multi-branch retailers. This would require introducing a location entity, updating the role model with location-level permissions, and adding cross-location reporting.

**Payment Gateway Integration:** Integrating with mobile money APIs (MTN MoMo, Airtel Money) and card payment processors (Stripe, Flutterwave) would enable real-time payment processing with automatic reconciliation, reducing the risk of cash handling errors and accelerating checkout.

**Supplier and Purchase Order Management:** Adding supplier management, purchase order creation and tracking, and automated purchase order generation based on low-stock thresholds would complete the inventory management lifecycle from procurement to point of sale.

**Hardware Integration:** Enhancing the system with native barcode scanner support through the Web Serial API or Hardware Events API, receipt printer integration via WebUSB or network printing, and customer-facing display support would provide a more complete POS hardware ecosystem.

**Advanced Analytics and Machine Learning:** Implementing predictive analytics for demand forecasting, seasonal trend analysis, automated reorder suggestions, and anomaly detection in sales patterns would provide additional business intelligence value.

**Mobile Application:** Developing dedicated mobile applications for Android and iOS would provide better offline support, access to device features such as camera-based

barcode scanning and NFC payment acceptance, and optimised touch interactions for phone-sized screens.

**Multi-Currency Support:** Adding support for multiple currencies with configurable exchange rates and automatic conversion would serve businesses operating across borders or serving international customers, particularly relevant for East African tourist destinations.

## Bibliography

- [1] Meta Platforms, Inc. React 19 Documentation. <https://react.dev>, 2025.
- [2] OpenJS Foundation. Express.js. <https://expressjs.com>, 2025.
- [3] The PostgreSQL Global Development Group. PostgreSQL Documentation. <https://postgresql.org/docs>, 2025.
- [4] Drizzle Team. Drizzle ORM Documentation. <https://orm.drizzle.team>, 2025.
- [5] Vercel Inc. Vercel Documentation. <https://vercel.com/docs>, 2025.
- [6] Neon Inc. Neon Serverless PostgreSQL. <https://neon.tech/docs>, 2025.
- [7] Michael McLaughlin. decimal.js: An Arbitrary Precision Decimal Type. <https://github.com/MikeMcl/decimal.js>, 2025.
- [8] Colin McDonnell. Zod: TypeScript-First Schema Validation. <https://zod.dev>, 2025.
- [9] Better Auth. Better Auth Documentation. <https://better-auth.com>, 2025.
- [10] Tailwind Labs. Tailwind CSS v4 Documentation. <https://tailwindcss.com/docs>, 2025.
- [11] Paul Henschel. Zustand: Bear-Themed State Management. <https://github.com/pmndrs/zustand>, 2025.
- [12] Google. Playwright: Cross-Browser Automation. <https://playwright.dev>, 2025.
- [13] Dave Cohen and Carlos Matos. Third Year Projects: Rules and Guidelines. Isbat University Kampala, 2013.
- [14] Vitest Team. Vitest: Blazing Fast Unit Test Framework. <https://vitest.dev>, 2025.
- [15] ApexCharts. ApexCharts.js Documentation. <https://apexcharts.com/docs>, 2025.
- [16] Drizzle Team. Drizzle Kit: Schema Migration Tool. <https://orm.drizzle.team/kit-docs/overview>, 2025.
- [17] Evan You. Vite: Next Generation Frontend Tooling. <https://vite.dev>, 2025.
- [18] Remix Team. React Router v7 Documentation. <https://reactrouter.com>, 2025.
- [19] Nico Hartzler. tsx: TypeScript Execution for Node.js. <https://github.com/privatenumber/tsx>, 2025.
- [20] OpenJS Foundation. Node.js Documentation. <https://nodejs.org/docs>, 2025.